

Mini Game Hub

Larissa
Sahil Deo

October 1, 2026

Contents

1	Security: Bash-Specific Vulnerabilities in Authentication	3
1.1	Option injection into <code>echo</code> (found and fixed)	3
1.2	Input that <code>read</code> silently rewrites	3
1.3	Pattern and record injection through usernames	3
1.4	Checking the password	4
1.5	Other Bash hazards	4
1.6	Summary and remaining limitations	4
2	External Libraries	5
3	Design & Architecture	5
3.1	Authentication System (Bash)	5
3.2	Leaderboard System	5
3.3	Game Class Abstraction	5
3.4	Rendering and Scaling	6
3.5	Event Handling and Game Loop	6
3.6	Win Detection	6
3.7	Animations	6
3.8	Blitz Mode	6
3.9	User Interface Components	7
3.10	Othello Move Logic	7
3.11	Connect 4 Mechanics	7
3.12	Flowchart:	7
4	Features Implemented	7
4.1	Authentication System	7
4.2	Blitz Feature	8
4.3	Animations	8
4.4	Performance Optimization	8
4.5	Abstractions	8
5	Directory Structure	9
6	Installation & Setup	9

7	Usage	9
8	Retrospective	11
9	What we would do differently with 10 more hours	12
9.1	Code Architecture Improvements	12
9.2	User Experience Enhancements	12
9.3	Feature Expansion	13

1 Security: Bash-Specific Vulnerabilities in Authentication

The login in `main.sh` reads untrusted text from the keyboard and splices it into `grep` and `sed` patterns and into the tab- and comma-separated files that store accounts and lockouts. Bash has several quirks that turn this kind of code into a security problem: it can silently rewrite input, treat data as command options, or treat data as a pattern. This section lists each risk we considered, how the code deals with it, and one rare bug we found and fixed while reviewing our own code.

1.1 Option injection into echo (found and fixed)

Passwords were hashed with

```
hash(){ echo -n "$@" | sha256sum | cut -d ' ' -f 1; }
```

The `-n` was there to stop `echo` adding a trailing newline, which would change the hash. The flaw is that Bash's `echo` does not accept `--` to end its options, so it reads its *argument* as more options whenever that argument looks like one [1]. A password of `-n`, `-e` or `-E` (or combinations such as `-ne`) is swallowed as a flag, nothing is printed, and the password is stored as the SHA-256 hash of the *empty string*:

```
$ echo -n "-n" | sha256sum      # e3b0c442... (same as the empty string)
$ echo -n ""    | sha256sum      # e3b0c442...
$ printf '%s' "-n" | sha256sum  # 5249f4fc... (correct)
```

Two things go wrong. All of these passwords collide with each other, and an account whose password is `-n` can be entered by pressing Enter at the password prompt. The fix is to print with `printf`, whose format string is fixed, so the password is only ever data:

```
hash(){ printf '%s' "$*" | sha256sum | cut -d ' ' -f 1; }
```

1.2 Input that read silently rewrites

- **Backslashes.** Plain `read` treats a backslash as an escape character and deletes it, so the password `a\b` would be stored as `ab`. Every `read` uses `-r`, which keeps backslashes literal.
- **Leading and trailing spaces (fixed).** Even with `-r`, `read` trims leading and trailing spaces and tabs using IFS, so `" pw "` and `"pw"` were the same password. Password prompts now use `IFS= read -r -s`, which keeps every character.
- **Echoing.** Passwords are read with `-s`, so they are never shown on screen.

1.3 Pattern and record injection through usernames

The username is placed directly inside regular expressions and a `sed` command:

```
grep -q  "^${name}${tab}"      .user_files/.users.tsv
grep -E  "^${username},[0-9]+$" .system_files/.failed_logins.csv
sed -i   "/^$line$/d"          .system_files/.failed_logins.csv
```

A username such as `.*` would match other users' rows, a `/` would break out of the `sed` address, and a tab, comma or newline would let a user write forged rows into the account or lockout files. We prevent all of these by accepting only usernames that match `^[a-zA-Z0-9_]+$`. Such a name contains no regex metacharacters, no `sed` delimiter and none of the file delimiters, so it can be inserted into these patterns safely.

1.4 Checking the password

The credential check builds the line `username<TAB>hash` and looks for it with

```
grep -Fxq "$line" .user_files/.users.tsv
```

`-F` treats the line as a fixed string rather than a regex, and `-x` requires the whole line to match [2], so no partial or prefix match can authenticate a user. Because only the 64-character hexadecimal hash is ever stored or compared, special characters in a password (quotes, backslashes, tabs, `$`) can never reach a file or a pattern.

1.5 Other Bash hazards

- **Word splitting and globbing.** Every expansion of user input is double-quoted (`"$u1"`, `"$pwd"`), so a `*` or a space in input is never expanded into file names or split into several arguments.
- **Passwords in the process list.** `printf` is a Bash builtin and the password reaches `sha256sum` through a pipe, so it never appears as a command-line argument visible to other users through `ps`.
- **Brute force.** After three wrong passwords the account is locked for 60 seconds, with the lock time recorded in `.system_files/.failed_logins.csv`.

1.6 Summary and remaining limitations

Risk	Effect if unhandled	Defence
Option injection into <code>echo</code>	Passwords <code>-n</code> , <code>-e</code> , <code>-E</code> hash as the empty string	<code>printf '%s'</code> (fixed)
Backslash escapes in <code>read</code>	Backslashes silently removed	<code>read -r</code>
Whitespace trimming in <code>read</code>	Leading/trailing spaces ignored	<code>IFS= read -r</code> (fixed)
Regex / <code>sed</code> injection	Matching or deleting other users' rows	Username whitelist
Delimiter injection	Forged account or lockout rows	Username whitelist; only hashes stored
Partial matches	Logging in on a prefix match	<code>grep -Fx</code>
Word splitting, globbing	Input expanded or split	Quoting every expansion
Password guessing	Unlimited attempts	60-second lockout

Two weaknesses remain and are listed under future work: the hashes are unsalted SHA-256, which is fast to brute-force if `.users.tsv` leaks (a salted, deliberately slow hash such as `bcrypt` or `Argon2` would be better), and the data files are hidden but not protected by file permissions.

2 External Libraries

- **numpy** - Used for efficient handling of arrays and game board matrix. It simplifies operations on the game board, such as checking valid moves, updating states, and checking the win condition.
- **pygame-ce** - Used for building the graphical user interface of the games. It handles rendering, mouse detection, game loops, and animations.
- **matplotlib** - Used for visualizing game statistics such as top 5 winners and the frequencies of each game
- **time** - Used for implementing delays, timers, and tracking time-based events within games.
- **os** - Used for interacting with the file system via bash (`os.system()`)
- **sys** - Used for taking command line arguments for `game.py`

3 Design & Architecture

3.1 Authentication System (Bash)

User authentication is handled via shell scripts:

- User credentials are stored in `.users.tsv` with SHA-256 hashed passwords.
- Login allows 3 attempts before temporary lockout.
- Lockout timestamps are stored in `.failed_logins.csv`.
- Defences against Bash-specific attacks are described in Section 1.

3.2 Leaderboard System

Game statistics are stored in CSV files. The leaderboard script:

- Sorts users by wins, losses, or win/loss ratio
- Formats output using `column`
- Maintains a top-5 players file

3.3 Game Class Abstraction

The `Game` class stores:

- The board as a NumPy 2D array
- Current player turn (boolean)
- Board dimensions and cell size

It provides utility functions such as `switchTurn()`, `valid_move()`, and a pluggable `checkWin()` function passed during initialization, allowing reuse across different games.

3.4 Rendering and Scaling

All rendering is done using `pygame`. Custom scaling functions `scale_h()` and `scale_w()` ensures resolution-independent scaling by mapping coordinates relative to a reference screen size.

Static elements such as boards are pre-rendered onto `Surface` objects and blitted every frame to improve performance.

3.5 Event Handling and Game Loop

Each game runs a main loop that:

- Captures user input (mouse clicks, key presses)
- Updates game state (moves, turn switching, timers)
- Renders updated visuals

Mouse clicks are mapped to board cells, and moves are executed only if valid.

3.6 Win Detection

Each game defines its own `checkWin()`:

- Tic Tac Toe and Connect 4 scan in 4 directions (horizontal, vertical, both diagonals) using NumPy slicing.
- Othello determines the winner by counting pieces when no valid moves remain (using `np.sum()`)

3.7 Animations

Animations are time-based using timestamps:

- Tic Tac Toe: progressive drawing of crosses (line segments) and circles (arc sweep).
- Connect 4: falling discs simulated using $y = \frac{1}{2}gt^2$.
- Othello: flipping discs using `pygame.draw.ellipse()` and expansion.

Jagged drawing effects are implemented via the `JaggedLine` class, which perturbs line segments randomly for a hand-drawn aesthetic. `pygame.draw.arc()` is naturally jagged, which is what inspired the hand-drawn aesthetic.

3.8 Blitz Mode

A `Timer` class manages turn-based countdowns. Each player's timer activates only during their turn. The remaining time is displayed as a dynamically coloured vertical bar, transitioning from green to red as time decreases.

3.9 User Interface Components

Reusable UI elements include:

- **Button:** handles hover, click feedback, and rendering
- **Box:** used for popups and text displays

End-of-game overlays use semi-transparent surfaces to dim the screen and display options like “Play Again” and “Main Menu”.

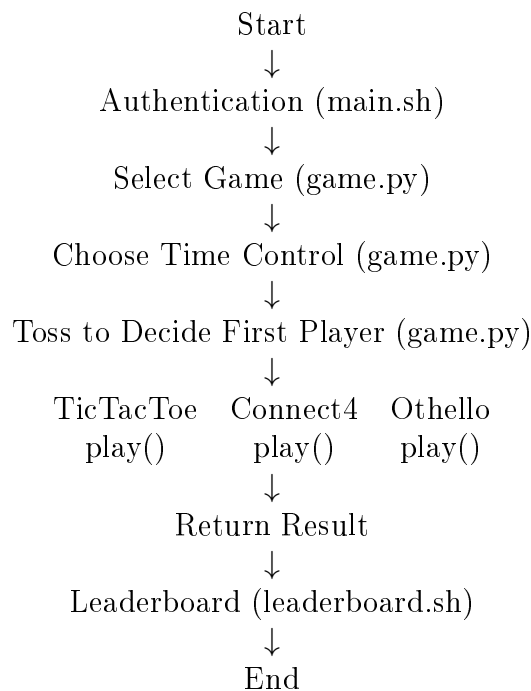
3.10 Othello Move Logic

Valid moves are computed by scanning in all 8 directions from each empty cell. A move is valid if it captures at least one opponent piece bracketed by the current player’s pieces. Piece flipping is performed by updating slices of the NumPy array.

3.11 Connect 4 Mechanics

Each column tracks the number of filled cells. When a move is made, the disc is placed in the lowest available row. Falling animation is applied until the disc reaches its final position.

3.12 Flowchart:



4 Features Implemented

4.1 Authentication System

- Supports user registration and login with passwords securely hashed using SHA-256.
- Two-player authentication is required before starting a game.

- The system enforces a 3-attempt login limit per user, after which a temporary time-based lockout is applied.

4.2 Blitz Feature

- A timed gameplay mode where each player must make a move within a specified time limit.
- The remaining time is visually represented using a vertical timer bar which changes colour according to the urgency of move.

4.3 Animations

Visual enhancements implemented to improve user experience:

- Tic Tac Toe: drawing animations for X and O symbols, along with a line animation highlighting the winning combination.
- Othello: coin flipping with a wave-like animation to represent piece conversion visually.
- Connect Four: a falling animation for coins to simulate realistic gravity-based placement.

4.4 Performance Optimization

- Using set in Othello to improve performance
- Storing data in .user_files to avoid processing from history.csv everytime - Time complexity improvement from $O(n \log n)$ to $O(u)$, where n is number of games and u is number of users
- Preprocessing static images/frames onto Surfaces and only blitting at runtime

4.5 Abstractions

- **Developer Point of View**
 - Design elements which are inheritable as a Class Button, Box, Jagged Line etc.
 - Menus - callable as a function, returns the choice
 - Auto-Sizing of Text - Using Binary Search text is automatically fit to Box/Button - we try a font, and get its rect - If it doesn't fit, we try a font size smaller (We do this using binary search between 1 to 100)
- **User Point of View**
 - GameHub folder is clean - system files and user files are hidden
 - Intuitive Menus and usage

5 Directory Structure

```
hub/
├── main.sh
├── game.py
├── leaderboard.sh
├── reset.sh
├── games/
│   ├── tictactoe.py
│   ├── connect4.py
│   ├── othello.py
│   ├── classGame.py
│   └── design_elements.py
├── images/
│   ├── red_disc.png
│   └── yellow_disc.png
├── .user_files/
│   ├── .stats_tictactoe.csv
│   ├── .stats_connect4.csv
│   ├── .stats_othello.csv
│   ├── .users.tsv
│   ├── .user_total_wins.csv
│   ├── .history.csv
│   ├── .top5.txt
│   └── .game_frequencies.csv
├── .system_files/
│   └── .failed_logins.csv
└── report/
    ├── report.tex
    ├── report.pdf
    ├── references.bib
    ├── makefile
    └── images/
```

6 Installation & Setup

```
pip install numpy pygame-ce matplotlib
```

7 Usage

Run the main script to start the application:

```
bash main.sh
```

Authentication

Two users are prompted to log in via the terminal. Each user has up to three attempts before a temporary lockout.

```
hp@LAPTOP-SE6687L5 MINGW64 ~/OneDrive/Desktop/COURSES/CS108 project
$ bash main.sh

--User1 Login--

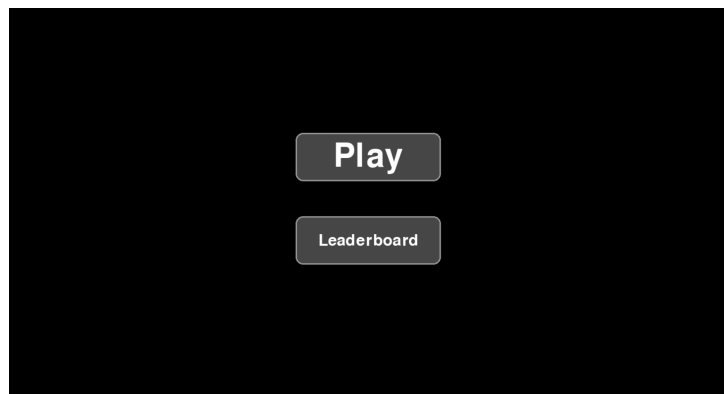
Enter username: a
Enter pwd:
User a Logged in successfully

--User2 Login--

Enter username: b
Enter pwd: |
```

Main Menu

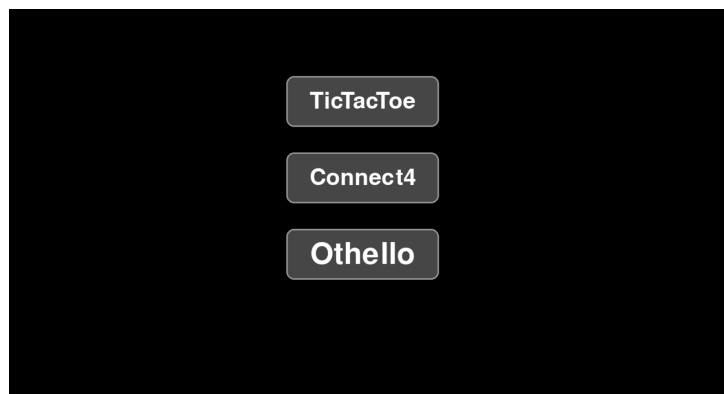
After launching, the main menu is displayed with options to start a game or view the leaderboard.



If play is clicked:

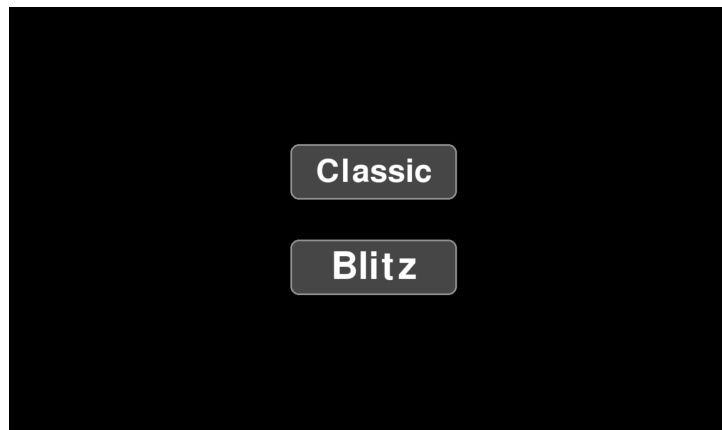
Choosing game

Click the button for game you want



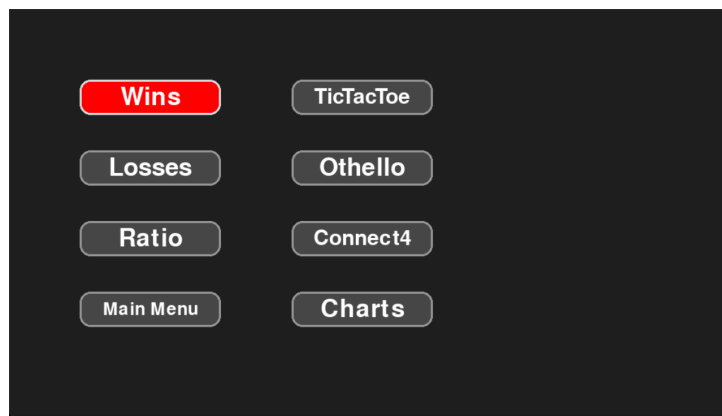
Choosing time control

Click the time control you want



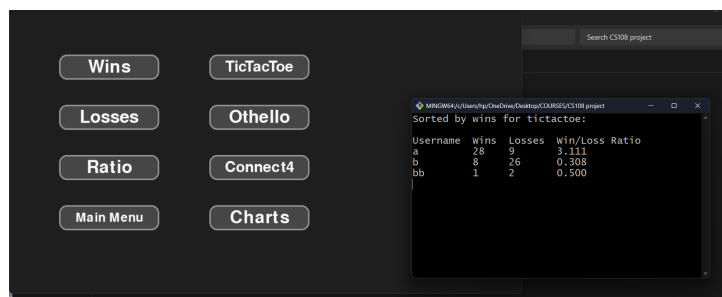
Leaderboard Menu

Click metric to sort by and game name for stats. The first choice is highlighted red until second is chosen



Leaderboard Output

The leaderboard is displayed in the terminal in a formatted table.



8 Retrospective

- Accidental change of the same part by both teammates -
Fix: Clearly communicating beforehand, and also doing a merge commit when necessary

- Lagging animation -
Fix: Performance improved by precomputing static frames onto Surface objects and then blitting them to the screen at loop runtime [9]
- Timer and Toss Bar fluid going out of box for low levels -
Fix: Using pygame Surfaces transparency feature - made a window Surface which lets viewer peek through only the shape we want, hence clipping the fluid [3]
- Scaling of sizes across systems -
Fix: Using scaling functions wherever necessary
- Not able to change game board values in Othello - the diagonal array extracted from game board using the `diagonal()` function was not writable by default.
Fix: Used `arr_name.setflags(write=1)` [4]
- Seed was same for all board lines since time was converted to int, and execution was finished in less than a second.
Fix: Used `int(time_ns())`
- Unnecessary complicated scaling by introducing a Size class -
Fix: Removed overcomplicated Size class, changed it to simple scaling functions
- Passwords `-n`, `-e` and `-E` were hashed as the empty string, because `echo -n "$@"` read them as options -
Fix: Hash with `printf '%s' "$@"` instead (Section 1.1) [1]
- Pygame Events during `toss()` like random clicks and ESCAPE were getting transferred to next loop -
Fix: `pygame.event.clear()`

9 What we would do differently with 10 more hours

9.1 Code Architecture Improvements

- Introduce a state-machine type flow for `game.py` to simplify the logic
- Create a Menu Class with Button and Box as subclass
- Use enums for things like whose turn is it to avoid all the confusion during development
- Replace unsalted SHA-256 with a salted, slow password hash (bcrypt or Argon2) and restrict permissions on the data files

9.2 User Experience Enhancements

- Add smoother scene transitions (fade in/out between menus and games).
- Improve aesthetics and feedback (hover effects, button animations, sound effects).
- Add a pause button

9.3 Feature Expansion

- Add single-player mode which just chooses random choices (Just for fun)
- Figure out a cleaner leaderboard display aesthetic so we can see terminal output simultaneously
- Create a visual interface to choose how many seconds per move in blitz

References

- [1] *GNU Bash Reference Manual: Bash Builtin Commands (echo, printf, read)*. URL: https://www.gnu.org/software/bash/manual/html_node/Bash-Builtins.html.
- [2] *GNU Grep Manual: Matching Control (-F, -x)*. URL: <https://www.gnu.org/software/grep/manual/grep.html#Matching-Control>.
- [3] *How to Draw a Semi-Transparent Circle in Pygame*. URL: <https://stackoverflow.com/questions/51229740/how-to-draw-a-semi-transparent-circle-in-pygame>.
- [4] *How to Make a NumPy Array Read-Only*. URL: <https://www.geeksforgeeks.org/python/how-to-make-a-numpy-array-read-only/>.
- [5] *Hyperlinks in LaTeX*. URL: <https://www.overleaf.com/learn/latex/Hyperlinks>.
- [6] *Making a Simple Directory Tree*. URL: <https://tex.stackexchange.com/questions/5073/making-a-simple-directory-tree>.
- [7] *NumPy Absolute Beginners Guide*. URL: https://numpy.org/doc/stable/user/absolute_beginners.html.
- [8] *Pygame Draw Module Documentation*. URL: <https://www.pygame.org/docs/ref/draw.html>.
- [9] *Pygame Surface Documentation*. URL: <https://www.pygame.org/docs/ref/surface.html#pygame.Surface>.
- [10] *Pygame Transform Smoothscale*. URL: <https://www.pygame.org/docs/ref/transform.html#pygame.transform.smoothscale>.
- [11] *Spines Demo*. URL: <https://matplotlib.org/stable/gallery/spines/spines.html>.
- [12] *Subplots Demo*. URL: https://matplotlib.org/stable/gallery/subplots_axes_and_figures/subplots_demo.html.